

선행 적재 & 간접 참조로 연출-게임플레이 전환 끊김 해결

1. 문제 / 설계 목표

내러티브 어드벤처 게임의 특성상, 한 사이클 안에서 시네마틱 시퀀스 → 대화 그래프 → 시간 진행 → 다음 시네마틱으로 연속 전환이 반복된다. 각 전환 시점에 다음 단계가 참조할 글로벌 데이터(UKausGameData의 유닛 스탯 테이블 등)가 메인 스레드에서 처음 로드되면 화면이 한순간 멈춘다.

설계 목표는 다음 두 가지였다:

- **컴파일-패키지 의존성을 헤더 단계에서 끌고 다니지 않기** — 글로벌 데이터를 참조하는 코드가 늘어나도 빌드 영향 면적을 일정하게 유지.
- **첫 화면 전 핵심 데이터를 미리 적재하고, 한 번 로드된 자산은 게임 종료까지 살아 있게 하기** — 실행 중 GC 수집으로 인한 재로드 비용을 0으로 만든다.

2. 구현 구조

핵심 파일:

- Source/Kaus/System/KausAssetManager.h
- Source/Kaus/System/KausAssetManager.cpp

세 가지 메커니즘이 한 클래스에 응집되어 있다.

2.1 Soft 참조 기반 일반 로더 (GetAsset 템플릿)

TSoftObjectPtr<AssetType>를 받아 동기 로드 후 반환한다. 호출 측은 자산에 대한 강참조를 헤더에 두지 않아도 된다.

```
template<typename AssetType>
AssetType* UKausAssetManager::GetAsset(const TSoftObjectPtr<AssetType>&
AssetPointer, bool bKeepInMemory)
{
    AssetType* LoadedAsset = nullptr;

    const FSoftObjectPath& AssetPath = AssetPointer.ToSoftObjectPath();

    if (AssetPath.IsValid())
    {
        // 이미 메모리에 있으면 동기 로드 비용을 피한다.
        LoadedAsset = AssetPointer.Get();
        if (!LoadedAsset)
        {
            // 강제 동기 로드 - 첫 접근 시점에만 발생.
            LoadedAsset = Cast<AssetType>(AssetPath.TryLoad());
            ensureAlwaysMsgf(LoadedAsset, TEXT("Failed to load asset [%s]"),
                *AssetPointer.ToString());
        }
    }
}
```

```

        if (LoadedAsset && bKeepInMemory)
        {
            // LoadedAssets 에 등록 → GC 방지. 게임 종료 시까지 살아 있다.
            Get().AddLoadedAsset(Cast<UObject>(LoadedAsset));
        }
    }

    return LoadedAsset;
}

```

2.2 부팅 시 선행 적재 (StartInitialLoading)

`UAssetManager::StartInitialLoading` 은 엔진이 모듈 초기화 직후 한 번 호출하는 훅이다. 여기서 `GetGameData()` 를 부르면 첫 화면이 뜨기 전 데이터가 이미 로드 완료된 상태가 된다.

```

void UKausAssetManager::StartInitialLoading()
{
    Super::StartInitialLoading();

    // 글로벌 데이터 자산을 미리 로드해 두어 진입 후 첫 접근에서 멈춤이 없게 한다.
    GetGameData();
}

```

2.3 영구 상주 캐시 (GetGameData + LoadedAssets TSet)

`LoadedAssets` 는 `UPROPERTY()` 로 선언된 `TSet<TObjectPtr<const UObject>>` — UE 의 GC 가 강참조로 인식 해 수집하지 않는다. 한 번 로드된 글로벌 데이터는 이 풀에 들어가 AssetManager 수명(=게임 수명) 동안 살아 있다.

```

const UKausGameData& UKausAssetManager::GetGameData()
{
    if (CachedGameData)
    {
        // 이미 로드된 경우 — 진입 후 정상 경로.
        return *CachedGameData;
    }

    UKausGameData* LoadedData = nullptr;
    if (!KausGameDataPath.IsNull())
    {
        // 부팅 시점이라 동기 로드가 허용된다. 한 번만 발생.
        LoadedData = KausGameDataPath.LoadSynchronous();
    }

    if (LoadedData)
    {
        CachedGameData = LoadedData;
        // LoadedAssets 에 강참조로 등록 → 게임 종료 시까지 GC 안전.
        LoadedAssets.Add(CachedGameData);
    }
}

```

```

    else
    {
        // 경로 오설정/누락 등으로 로드 실패 - 호출자에게 nullptr 을 강제하지 않도록 빈 인스턴스 fallback.
        UE_LOG(LogKaus, Error, TEXT("Failed to load KausGameData! Check DefaultEngine.ini"));
        CachedGameData = NewObject<UKausGameData>();
    }

    return *CachedGameData;
}

```

`AssetManagerClassName` 은 `Config/DefaultEngine.ini` 에서 `KausAssetManager` 로 지정되어, 엔진의 `GEngine->AssetManager` 가 이 인스턴스로 대체된다.

3. 핵심 설계 결정

Hard Reference 대신 Soft Reference 를 택한 이유. 글로벌 데이터를 사용하는 코드가 늘 때마다 헤더 의존성과 패키지 의존성이 함께 커지는 것을 막기 위함이다. `TSoftObjectPtr` 는 컴파일 단계에서 자산을 실제로 끌고 다니지 않으며, 호출 시점에만 `TryLoad()` 로 한 번 비용을 지불한다. 게임 진입 시에는 `StartInitialLoading` 에서 미리 로드해 두므로 게임 중에는 캐시 히트만 발생한다.

bKeepInMemory 플래그로 GC 를 차단한 이유. 글로벌 데이터(스택 테이블, 게임 데이터 자산)는 "한 번 로드되면 게임 종료까지 살아 있어야 한다" 라는 규약을 갖는다. 평상시 GC 가 자산을 수집해 다음 사용 시 다시 동기 로드가 일어나면 그것이 정확히 본 시스템이 해결하려는 화면 끊김 문제가 된다. 그래서 `LoadedAssets` 라는 별도 `UPROPERTY` set 에 강참조로 등록한다. UE 의 GC 는 이 set 을 통해 객체에 도달할 수 있으므로 수집 대상에서 제외한다. 한 번만 쓰고 버릴 자산을 위해 `bKeepInMemory=false` 옵션도 남겨 두었다.

로드 실패 시 빈 NewObject 로 fallback 한 이유. `GetGameData()` 는 `const UKausGameData&` 를 반환한다. 호출 측이 null 체크를 강제당하지 않게 하려는 의도다. ini 경로 오설정 같은 환경 문제로 로드가 실패해도 빈 인스턴스를 반환해 호출 코드가 그대로 동작하게 하고, 에러는 별도 로그로 명시한다.

4. 결과 / 얻은 것

- 글로벌 데이터를 참조하는 모든 코드가 헤더에 강참조 없이 동작 — 빌드 의존성 면적 일정 유지.
- 게임 진입 후 글로벌 데이터 접근은 모두 캐시 히트 — 첫 접근 비용은 부팅 시점에 한 번만 발생.
- `LoadedAssets` 풀로 한 번 로드된 자산이 의도치 않게 GC 수집되는 사고가 원천 차단됨.
- 같은 패턴(`GetAsset` / `GetSubclass` 템플릿)이 다른 자산 종류에도 그대로 적용 가능 — 팀이 새 글로벌 자산을 추가할 때 동일한 규약으로 작업.

관련 파일

파일	역할
<code>System/KausAssetManager.h</code>	클래스 선언 + 템플릿 정의 (<code>GetAsset</code> / <code>GetSubclass</code>)
<code>System/KausAssetManager.cpp</code>	싱글톤 접근, <code>StartInitialLoading</code> , <code>GetGameData</code> 구현
<code>Config/DefaultEngine.ini</code>	<code>AssetManagerClassName</code> 오버라이드 지정